React JS Interview Questions

Complete Merged Edition — 125 Questions | 13 Sections

SECTION 1 — REACT FUNDAMENTALS

1. What is React?

React is an open-source JavaScript library used for building user interfaces (UI). It simplifies creation of single page applications by using reusable components.

2. What are the key features of React JS?

- Virtual DOM
- Component-based architecture
- Reusability
- JSX
- Declarative syntax
- React Hooks
- One-way Data Binding

3. What is DOM? What is the difference between HTML and DOM?

DOM stands for Document Object Model. It represents the web page as a tree-like structure which allows JavaScript to dynamically access and manipulate the content and structure of a web page. HTML is the markup language used to write the page; the DOM is the in-memory object representation of it.

4. What is Virtual DOM? Difference between DOM and Virtual DOM?

DOM	Virtual DOM
Actual representation of the web page	Lightweight copy of the DOM
Re-renders entire page on update	Re-renders only the changed parts
Can be slow with frequent updates	Optimised for fast rendering

5. What are React Components? What are the types?

React components are reusable building blocks for creating user interfaces. There are 2 types: 1. Class Components 2. Functional Components

6. What is a Single Page Application (SPA)?

It does not load the entire page every time. When a user clicks on a component, only that component is loaded/updated without a full page refresh.

7. What are the advantages of React?

- Simple to build single page applications using components
- Cross-platform and open source
- Lightweight and very fast
- Uses Virtual DOM
- Large community and ecosystem

- Time saving, allows faster development
- Allows use of third-party libraries

8. What is the role of JSX in React?

- JSX stands for JavaScript XML.
- JSX is used by React to write HTML-like code inside JavaScript.
- JSX is converted to JavaScript via tools like Babel, because browsers understand JavaScript, not JSX.

9. What is the difference between Declarative and Imperative syntax?

Declarative syntax (React/JSX): focuses on describing the desired result without specifying the step-by-step process. Imperative syntax (plain JavaScript): involves a step-by-step process to achieve a particular goal.

10. What is the Spread Operator?

The spread operator (...) is used to expand or spread an array or an object. It can be used to pass one array's values into another array or to copy/merge objects.

11. Is React a Framework or a Library? What is the difference?

React is a JavaScript Library. Developers import it and use its functions inside components. A Framework (e.g., Angular) requires developers to follow a specific structure or pattern defined by the framework.

12. What makes React unique from other frameworks?

It uses Virtual DOM which makes rendering faster and smoother. React is simpler to learn and more popular than Angular. React is smaller and lighter because it is a library, not a full framework.

SECTION 2 — REACT PROJECT SETUP

13. How to set up a React project?

- Install Node.js — https://nodejs.org
- Install a code editor (e.g., VS Code)
- Create a React project: npx create-react-app my-app
- Run the app: npm start

14. What are the main files in a React project?

- index.html — Single page for the React application
- Components/component1.js — Application components
- App.js — Main/root component
- App.test.js (Optional) — Tests for App.js
- index.css (Optional) — Global stylesheet
- index.js — Entry point; renders the root React component into the DOM

15. What is NPM? What is the role of the node_modules folder?

NPM (Node Package Manager) is used to manage the dependencies for a React project, including the React library itself. The node_modules folder contains all the dependencies of the project, including the React libraries.

16. What is the difference between npm and npx?

npm — installs packages and runs scripts. npx — runs a package directly without installing it globally.

17. What is the role of index.js and ReactDOM in React?

ReactDOM is a JavaScript library that renders components to the browser DOM. The index.js file replaces the root element of index.html with the newly rendered React components.

18. What is the role of export default inside App.js?

The export statement makes a component available for import using an import statement in other files.

19. Does the file name and the component name have to be the same in React?

No, they can be different. However, for better code readability and understanding, it is a good practice to keep them the same.

SECTION 3 — JSX & BABEL

20. What are the advantages of JSX?

- Improves code readability and writability
- Error checking in advance
- Supports JavaScript extensions

21. What is Babel?

Babel is a transpiler used in React to convert (transpile) JSX syntax into regular JavaScript that the browser can understand. Browsers can only understand JavaScript, HTML, and CSS — not JSX.

22. What is a Transpiler? Difference between Compiler and Transpiler?

Transpiler: converts high-level language code (e.g., JSX) to another high-level language (e.g., JavaScript). Example: Babel. Compiler: converts high-level language to low-level byte code.

23. Can a browser read a JSX file?

No. Browsers cannot directly interpret or understand JSX files. Babel takes JSX and converts it into equivalent JavaScript code that browsers can understand.

24. What is the role of Fragment in JSX?

Fragment (<>) is a way to group multiple children elements in React without adding unnecessary nodes to the DOM.

25. What are the types of Conditional Rendering in JSX?

- If/else statements
- Ternary Operator
- && Operator
- Switch Statements

26. How do you iterate over a list in JSX? What is the map() method?

The map() method allows iterating over an array and modifying its elements using a callback function. Example: const data = [2, 4, 6, 8, 10]; return data.map((num) => num * 2);

27. Does React use HTML?

No. React does not use HTML directly. Instead it uses JSX, which is similar to HTML but is converted to JavaScript.

SECTION 4 — COMPONENTS

28. What are Functional Components?

Functional components are declared as JavaScript functions. They are stateless by default but can manage state with the help of React Hooks.

29. What are Class Components in React?

Class components are defined using JavaScript ES6 classes. They are stateful components and use lifecycle methods. The render() method in a class component is responsible for returning JSX.

30. What is the role of the this keyword in class components?

The this keyword refers to the instance of the class, allowing access to its properties and methods.

31. Differences between Functional Components and Class Components?

Feature	Functional Component	Class Component
Syntax	Defined as a JS function	Defined as a JS(ES6) class
State	Stateless but can use hooks	Manages state with this.state
Lifecycle methods	No (uses hooks instead)	Yes
Readability	More readable & concise	Verbose (complex)
this keyword	No	Yes (Access props using this.props)

32. What are refs in React JS used for?

Refs are React functions used for accessing the DOM to change the value of a child component without using any other tool (like state).

33. What are Pure Components in React?

Pure components can replace any component using only the render() method. They implement a shallow comparison of props and state to decide whether to re-render, improving performance.

34. What are Render Props?

It is the technique of sharing code between components by passing a function as a component prop. This function is used to compose new components using the passed component.

35. What are keys in React used for?

Keys in React are used to identify unique Virtual DOM elements, helping React efficiently update and re-render lists.

36. Can props be updated in React JS?

No. Props cannot be updated in React JS — they are read-only (immutable). They are passed from parent to child and cannot be changed by the child.

37. What is Lifting State Up?

Lifting state up means moving shared state to the closest common ancestor component, so multiple child components can access and share that state via props.

SECTION 5 — PROPS, STATE & DATA FLOW

38. What are Props in JSX?

Props (short for Properties) are used to pass data from a parent component to a child component. They are read-only.

39. What is Prop Drilling in React?

Prop drilling is the process of passing props through multiple layers of components. To avoid it, use Redux or the Context API (useContext).

40. How do you pass data between functional components in React?

By using props — passing data from a parent component to a child component.

41. How to pass data from a child component to a parent component?

The parent provides a callback function to the child component. The child can then invoke this callback to pass data back to the parent.

42. Difference between State and Props?

STATE	PROPS
Internal data that a component manages	External data passed from parent
Mutable (can be changed and updated)	Immutable (cannot be changed)
Used to store data that changes over time	Used to pass data/configuration to child

43. What are State, Stateless, Stateful, and State Management terms?

State: refers to the current data of the component. Stateful / State management: when a user performs actions on the UI, the React application updates and re-renders that data/state on the UI. Stateless: a component that does not manage its own state.

44. What are Stateful and Stateless Components?

Stateful Component	Stateless Component
Has its own state (useState or this.state)	Does not manage any internal state
Can trigger re-renders based on state changes	Renders based only on props
More complex — handles logic and state	Simpler, focuses on rendering UI
Form inputs, counters, dynamic content	Presentational/static displays

SECTION 6 — REACT HOOKS

45. What are React Hooks?

- useState — State
- useEffect — Side effects
- useMemo — Performance
- useReducer — Complex state
- useCallback — Memoization
- useRef — Refs
- useContext — Context
- useLayoutEffect — Synchronous side effects

46. What is the role of useState() hook? How does it work?

useState() enables functional components to manage state. It accepts the initial state value as a parameter and returns an array with 2 elements: the current state value and a setter function. Example: const [count, setCount] = useState(0);

47. What is the role of useEffect()? How does it work?

- useEffect with empty array [] — runs only once when the component mounts (componentDidMount).
- useEffect with dependencies — runs every time a specified state/prop changes (componentDidUpdate).
- Cleanup function inside useEffect — runs on component unmount (componentWillUnmount).

48. What is the Dependency Array in useEffect()?

useEffect(() => { ... }, [someState]); If any change occurs on the listed state/prop, useEffect will be called again. An empty array [] means it runs only once on mount.

49. What is the role of useContext() hook in React?

useContext() provides a way to pass data from a parent component to deeply nested child components without using props (avoiding prop drilling).

50. What is createContext()? What are Provider and Consumer properties?

createContext() returns an object with Provider and Consumer properties. Provider — responsible for providing the context value to all its child components. useContext() or Consumer — used to consume the context value in child components.

51. When to use useContext() instead of props?

- Theme switching
- Localization
- Centralized user preferences
- Notification system

52. What are the similarities between useState() and useReducer()?

- Both are React hooks for managing state.
- Both trigger component re-render when state changes.
- Both store and update local component state.

53. What is useReducer()? When to use useState() vs useReducer()?

Use useState() when state is simple, updates are independent, and logic is small. Use useReducer() when state is complex (multiple values/nested objects), updates depend on previous state, or you have multiple actions (add, remove, update).

54. Difference between useState() and useReducer()?

useState	useReducer
Simple state	Complex state
Directly update with setter	Update using actions
Easy to use	Better structure for big logic
Good for small components	Good for large components

55. What are dispatch and reducer functions in useReducer?

reducer function: a pure function that decides how state changes based on an action. Example: (state, action) => newState. dispatch function: used to send actions to the reducer. Example: dispatch({ type: 'increment' })

56. Explain useMemo hook and give an example.

useMemo memoizes the result of an expensive calculation so it is only recomputed when its dependencies change. Example: const result = useMemo(() => compute(num), [num]);

57. Explain useCallback hook and give an example.

useCallback memoizes a function reference to prevent unnecessary recreations on re-renders. Example: const handleClick = useCallback(() => { console.log('Clicked!'); }, []);

58. Explain useRef hook and give an example.

useRef references DOM elements or persists values across renders without triggering re-renders. Example: const inputRef = useRef(null); // Later: inputRef.current.focus();

59. Explain the purpose of the useMemo hook.

useMemo is used by functional components to memoize expensive functions. These functions are only recomputed when the specified input (dependency) changes.

60. What are custom hooks?

Custom hooks are user-defined hooks that let you extract and reuse stateful logic across multiple components. They follow the naming convention use*.

SECTION 7 — COMPONENT LIFECYCLE

61. What are Component Lifecycle Phases?

- Mounting Phase — Component is created and inserted into the DOM.
- Updating Phase — Component re-renders due to state/prop changes.
- Unmounting Phase — Component is removed from the DOM.

62. What are Component Lifecycle Methods?

Mounting: Constructor → getDerivedStateFromProps → render() → componentDidMount() Updating: getDerivedStateFromProps → shouldComponentUpdate → render() → getSnapshotBeforeUpdate → componentDidUpdate() Unmounting: componentWillUnmount()

63. What is the role of the Constructor in class components?

The constructor is a special method called when an instance of a class is created. It is used for initializing the component's state or performing setup before the component is rendered.

64. What is the role of the super keyword in the constructor?

The super keyword is used in the constructor of a class component to call the constructor of the parent class, ensuring the parent's initialization logic is executed.

65. What is the role of the render() method in the component lifecycle?

The render() method returns the React elements that will be rendered to the DOM.

66. What is the role of componentDidMount()?

componentDidMount() runs after the component has been rendered. It is commonly used for side effects such as loading data from an external API.

67. How is state maintained in a class component?

Two-step process: 1. this.setState() — used to update the state. 2. this.state — used to render/access the updated state in the DOM.

68. What is Mounting and Unmounting in React JS?

Mounting: When elements are attached to the DOM. Unmounting (Demounting): When elements are detached from the DOM.

SECTION 8 — CONTROLLED & UNCONTROLLED COMPONENTS

69. What are Controlled Components in React?

A controlled component is a component whose form elements (like inputs or checkboxes) are controlled by the state of the application. The state is the single source of truth.

70. What are Uncontrolled Components?

In uncontrolled components, form element values are not controlled by React state. They typically use useRef() to access form element values directly from the DOM.

71. Differences between Controlled and Uncontrolled Components?

Controlled	Uncontrolled
Values controlled by React state	Values not controlled by React state
Event handlers update state	Values accessed directly via DOM
Does not use useRef()	Commonly uses useRef()
Re-renders on state changes	Less re-rendering
Recommended for form handling	Useful in certain scenarios

72. How to handle forms in React?

The preferred and recommended approach for handling forms in React is by using controlled components.

73. How can you handle multiple input fields in a controlled form?

Maintain separate state variables for each input field and update them individually using the onChange event.

74. What are Synthetic Events in React?

Synthetic events are React's cross-browser wrapper around native browser events. Examples: onChange, onClick.

SECTION 9 — ROUTING

75. What is Routing and React Router?

Routing allows you to create a single page web application with navigation without needing a full page refresh. React Router is a library for handling routing — it enables navigation and rendering of different components based on the URL.

76. What is React Router DOM?

React Router DOM is a library used to route from one component to another component in web applications.

77. What are the roles of Route and Routes in React routing?

Each route path is defined inside . Example: } />

78. What is the role of the Switch component in React routing?

The Switch component ensures that only the first matching route is rendered; the rest are ignored.

79. What is the role of the exact prop in React routing?

The exact prop is used with the Route component to match exactly the provided path.

SECTION 10 — STATE MANAGEMENT

80. What are the top 3 ways to achieve state management in React?

- useState Hook — for simple component-level state.
- Context API — for avoiding prop drilling when sharing global data.
- Redux — for large-scale applications with complex state; provides a centralized store.

81. Explain Redux.

- Store — stores the entire application state in one place
- Actions — plain objects describing what happened
- Reducers — pure functions that determine how state changes based on actions
- Dispatch — sends actions to the reducer

82. What is Redux Thunk?

Redux Thunk is a middleware in React (Redux) that enables users to create action creators that return a function instead of an action object. This allows for asynchronous operations in Redux.

83. What are the types of middleware in Redux?

There are two types: Thunk and Saga.

84. What is Flux in React?

Flux is an architectural pattern that ensures the one-way flow of data in a React application.

85. What is the difference between Flux and Redux?

Flux	Redux
Store has state and change logic	State and change logic are separate
Multiple stores	Only one store
All stores are disconnected and flat	Single store with hierarchical reducers
State is mutable	State is immutable

86. Explain Dispatch in Redux.

Dispatch is the central hub of the app where actions are sent. It stores the response data in a payload which can then be used in any component.

87. Explain the Store in Redux.

The store allows you to save the application's entire state in one place, making it accessible from any component.

SECTION 11 — CODE SPLITTING & PERFORMANCE

88. What is Code Splitting in React?

Code splitting is a technique to split the JavaScript bundle into smaller chunks that are loaded on demand, improving initial load performance.

89. How to implement Code Splitting in React?

- Use React.lazy() to lazily import components.
- Wrap components with to handle loading state.
- Configure the build tool (e.g., Webpack) for dynamic imports.

90. What is the role of Suspense and lazy in React?

React.lazy() — allows you to load a component asynchronously/dynamically only when it is needed. — displays a fallback UI while the lazily loaded component is being fetched.

91. What is Lazy Loading?

Lazy loading loads a particular component only when it is required, instead of loading all components upfront. Implemented using React.lazy() and .

92. What are the Pros and Cons of Code Splitting?

Pros: Faster initial load time, optimised bandwidth usage, improved browser caching, parallel loading of chunks, easier maintenance. Cons: Added complexity, tooling dependencies, potential for runtime errors, increased number of HTTP requests.

93. How can you optimise performance in a React application?

- useMemo and useCallback — memoize values and functions to reduce unnecessary recalculations.
- React.Fragment — avoid unnecessary wrapper elements.
- React.lazy — load components lazily to reduce initial bundle size.
- Code splitting — divide the app into smaller chunks loaded on demand.
- State management — use efficient state management to avoid unnecessary renders.

94. What is the use of React Profiler?

React Profiler is a set of tools in React that allows developers to profile (analyse) the performance of a React application, identifying components that re-render unnecessarily.

95. What is Higher Order Component (HOC) in React?

A Higher Order Component is a function that takes a component as an argument and returns a new component with added features/logic. It is an advanced technique for reusing component logic.

96. What are the 5 ways to style React components?

- Inline styles
- CSS style sheets
- CSS Modules
- Global style sheets
- CSS frameworks

97. What is the difference between fetch and axios for API calls?

fetch — a built-in JavaScript function, requires manual JSON parsing, no need for additional libraries. axios — a third-party library that simplifies HTTP requests and allows the use of interceptors for request/response logging, authentication, and error handling.

SECTION 12 — REACT ADVANCED TOPICS

98. What is Server-Side Rendering (SSR) vs Client-Side Rendering (CSR)?

SSR: The server creates the full webpage and sends it to the browser. The page shows up quickly and is good for SEO. CSR: The browser gets a basic page, then builds the rest using JavaScript. It takes longer initially but can be faster to use once loaded.

99. What is GraphQL?

GraphQL is a query language for APIs and a runtime for executing those queries. React components can use GraphQL queries to fetch the data required for rendering.

100. What are the differences between React and React Native?

React	React Native
JavaScript library	Framework
Builds web interfaces	Builds mobile applications
Runs on web browsers	Runs on iOS and Android
Uses HTML and CSS for UI	Uses native UI components (View, Text)
Deployed as web apps	Deployed through app stores

101. What are Error Boundaries in React?

Error boundaries are React components that detect JavaScript errors anywhere in the child component tree, log those errors, and display a fallback UI instead of crashing the entire component tree.

102. What is React Router? (Detail)

React Router is a routing library that allows the user to add new screens and flows to the application. It keeps the URL in sync with what is being displayed on the screen and permits switching between views without a full page reload.

103. What are popular Testing Libraries for React?

- Jest
- Cypress
- React Testing Library
- Enzyme

104. How can you implement Authentication in a React application?

- Frontend sends POST request with {username, password} to backend.
- Backend authenticates and creates a JWT Token.
- Backend returns the JWT Token in the response.
- Frontend stores the JWT Token in local storage.
- Future requests include the JWT Token in the Authorization header.
- Backend validates the token signature before responding with data.

105. What are the 5 ways to style React components? Explain inline styles.

Inline styles — applied directly using the style attribute with a JavaScript object. Other options: CSS style sheets, CSS Modules, Global style sheets, CSS frameworks.

106. Does React use any animation package?

Yes. React has two popular animation packages: react-motion and React Transition Group.

SECTION 13 — JAVASCRIPT ESSENTIALS FOR REACT

107. What is Hoisting in JavaScript?

Hoisting is the default behaviour of JavaScript where all variable and function declarations are moved to the top of their scope before code execution.

108. Difference between var, let, and const?

var — function-scoped; can be re-declared and updated. let — block-scoped; value can be changed but variable cannot be re-declared. const — block-scoped; value cannot be changed (constant).

109. What are block-level and function-level data types?

let and const — block-level data types. var — function-level data type.

110. ES6 features in JavaScript?

- Block scope with let and const
- Arrow functions
- Template literals

- Default parameters
- Spread operator
- Destructuring
- Classes
- Promises

111. What are Closures?

A closure occurs when an inner function has access to the variables of its outer (parent) function, even after the outer function has returned. This is because the inner function retains a reference to the outer function's scope.

112. Difference between map(), filter(), find(), and reduce()?

- map() — iterates over an array and returns a new array with transformed elements.
- filter() — filters elements based on a condition and returns a new array.
- find() — finds and returns the first element that matches a condition (returns a single element, not an array).
- reduce() — performs operations on an array and returns a single accumulated value.

113. Write an example of filter().

let data = [10, 12, 1, 3, 14, 15, 18, 21, 27, 31, 26, 33, 45]; const newData = data.filter((k) => k > 20); console.log(newData); // [21, 27, 31, 26, 33, 45]

114. What is a Promise in JavaScript?

A Promise follows asynchronous operation and has 3 states: - Pending — operation is still in progress - Resolved — operation completed successfully - Rejected — operation failed

115. What is Promise.all()?

Promise.all() is a JavaScript method that takes an array of Promises and returns a single Promise that resolves when all input Promises have resolved, or rejects if any one of them rejects.

116. Difference between Promise and a Callback?

Callback — a function passed as an argument to another function, called after an async operation. Can lead to 'callback hell'. Promise — a cleaner way to handle async operations with .then() and .catch(), and supports chaining.

117. What is the difference between synchronous and asynchronous?

Synchronous — follows a blocking mechanism; executes tasks one after another. Asynchronous — follows a non-blocking mechanism; can execute multiple tasks at once without waiting.

118. Difference between setInterval() and setTimeout()?

setInterval() — calls a function repeatedly at a specified interval. setTimeout() — executes a function once after a specified delay.

119. What is Shallow Copy vs Deep Copy?

Shallow copy — copies only the top-level structure of an object. If a change is made, both the original and the copy are affected. Deep copy — copies all levels and creates a completely new independent object.

120. How to check if a given string is a palindrome in JavaScript?

Convert the string to lowercase, then use split('').reverse().join('') to reverse it, and compare to the original. const isPalindrome = (str) => str.toLowerCase() === str.toLowerCase().split('').reverse().join('');

121. What is Array.reverse() in JavaScript?

Array.reverse() is used to reverse the order of elements in an array in place and returns the reversed array.

122. Difference between display: none and visibility: hidden?

display: none — the element does not appear and does not take up any space in the layout. visibility: hidden — the element is not visible but still occupies space in the layout.

123. What is Flexbox?

Flexbox (Flexible Box Layout) is a CSS layout model that provides an efficient way to lay out, align, and distribute space among items in a container, even when their size is unknown or dynamic.

124. Difference between session storage and local storage?

localStorage — persists data with no expiration; available across browser sessions. sessionStorage — stores data only for the duration of the browser session; cleared when the tab is closed.

125. What are CSS position attributes? Difference between fixed, absolute, and relative?

fixed — the element's position cannot be changed by scrolling; it is fixed relative to the viewport. absolute — positioned relative to its nearest positioned ancestor; does not affect other elements. relative — positioned relative to its normal position; does affect surrounding elements.